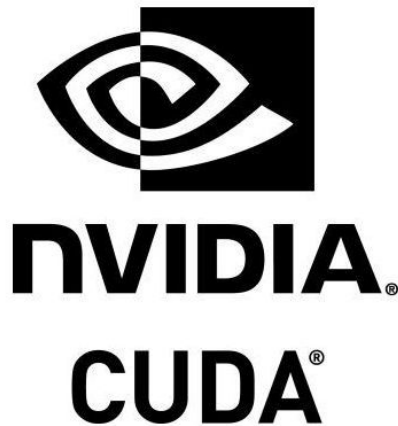




Introduction to CUDA

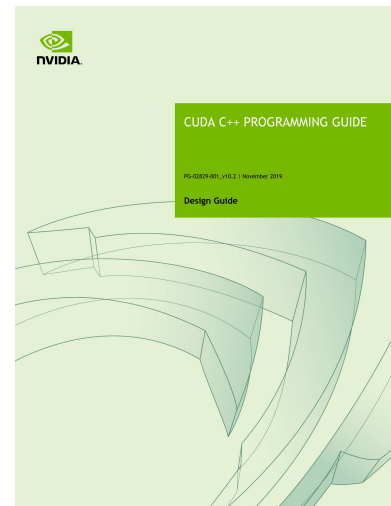
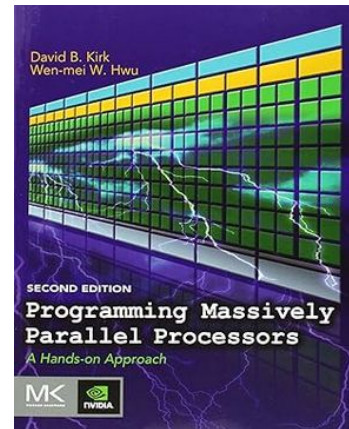
CS-6346.0U1
Written by Cody Hatfield

Overview

- What is CUDA?
- History and Applications
- Programming and Compiling
- Hardware and Software Architectures
- Best Practices
- Case Studies
 - Summation
 - Search
 - Simulation
- Special Topics

Literature Recommendations

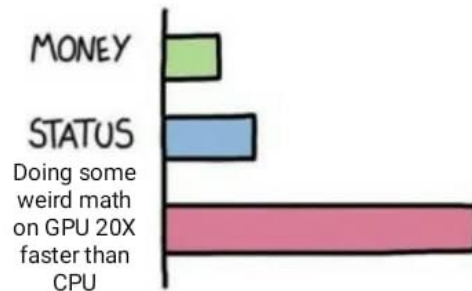
- Programming Massively Parallel Processors - A Hands-On Approach
- CUDA C++ Programming Guide - Design Guide *PG-02829-001_v10.2*
November 2019
- Online Reference Guide (not the same as the design guide)
 - <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>



What is CUDA?

- CUDA is the name of a language as well as the name of a parallel computing architecture.
 - You write CUDA code to run on CUDA hardware.
- Allows running arbitrary code on a large number of cores inside NVIDIA GPUs
 - The CUDA language looks and behaves like C/C++
- Can dramatically reduce compute times by parallelizing tasks.
 - Relatively easy to pick up and implement inside of C++ and Python based applications.

WHAT GIVES PEOPLE FEELINGS OF POWER



CUDA History

- Parallelized computing has existed since before digital computers were created.
- Early personal computers used parallel computing components known as GPUs to render and display media graphics.
- General-Purpose computing on Graphics Processing Units (GPGPU) was initially used though it is difficult to learn.
- CUDA was released in 2007.
- CUDA now has dominant market share, but AMD has begun to challenge them lately with ROCm.

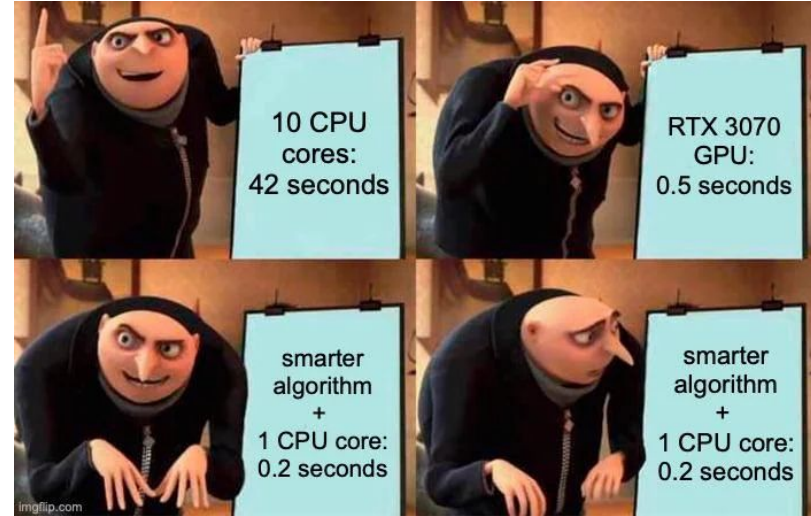
Common Applications

- Simulations
- Data Analysis
- Computer Vision
- Robotics



When to use CUDA

- Runtime can be significantly reduced if CUDA is used correctly.
- Clockspeeds on a GPU are about 20x slower than clockspeeds on a CPU.
- Memory on a GPU is often limited.
- CUDA is best used when a task must be repeated a large number of times.
 - Smarter algorithms may preclude this need, but not always.



How I've used CUDA

- Autonomous drones
- Vision processing
- Decision making
- Data generation
- Simulation



C++ vs CUDA

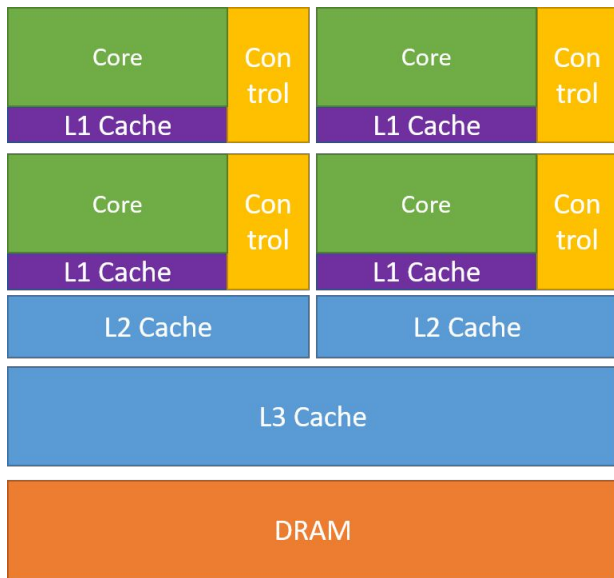
- CUDA is basically a slimmed-down version of C++ where many features have been removed, but the core functionality remains.
- Different compiler
- Most standard C++ libraries will not work in CUDA, however some standard library functions like “printf” are supported.
- Can't use classes in CUDA - everything must be procedural.
 - There are ways to get around this constraint.
- CUDA is intended to run inside a C++ or python application which drives the calls to the GPU

Hardware Configuration

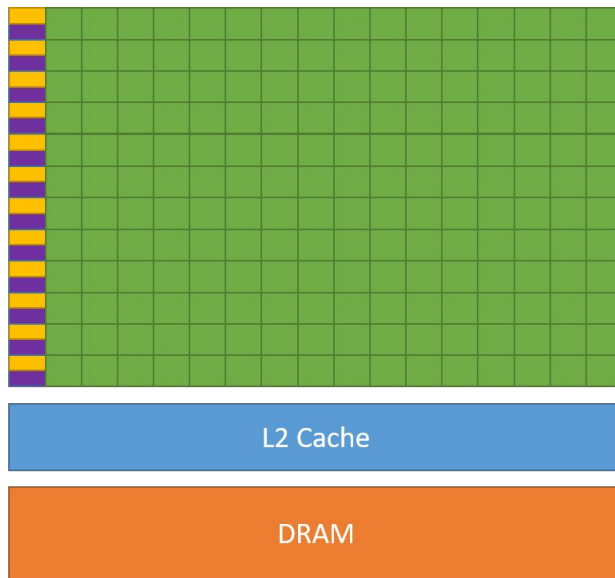
Each GPU is composed of many processing “cores”.

32 cores are always grouped into a single “warp”. When a program wants to use a core, it must reserve the whole warp.

There are shared caches inside of the warp as well as global caches for the entire GPU.



CPU



GPU

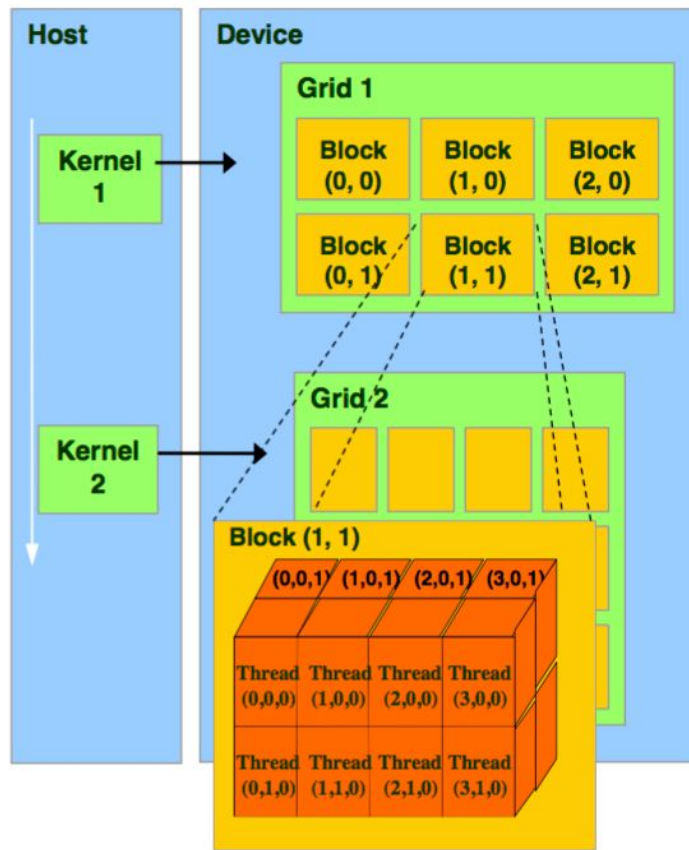
Software Configuration

Each CUDA program reserves a desired number of blocks on the GPU. When this reservation is made, the CUDA program also specifies how many threads will be active on the block.

Each block will be dispatched to an individual warp, and the threads allocated to the block will be distributed among the cores of the warp. If there are more threads than there are cores, then the cores will be shared by the threads as they are swapped in and out of memory.

Due to the computational overhead of context switching, you should **never** use more threads than there are cores. Since there are always exactly 32 cores for each warp on modern CUDA device (barring few exceptions) you should always use 32 threads or less in each block.

Each thread represents a single unit of work. If your code needs more than 32 units of work, you can spread your threads out across more blocks and still achieve the same result but with a faster runtime.



Compiling with nvcc

Makefile with a header-only approach (Linux):

```
all:
    nvcc -x cu -std=c++17 src/main.cu -lcuda -lcudart -I/opt/cuda/include -L/opt/cuda/lib64 -Wno-deprecated-gpu-targets -o main
```

- `nvcc`
- `-x cu -std=c++17`
- `src/main.cu`
- `-lcuda -lcudart`
- `-I/opt/cuda/include`
- `-L/opt/cuda/lib64`
- `-Wno-deprecated-gpu-targets`
- `-o main`

The nvidia cuda compiler (nvcc) is essentially a drop-in replacement for g++.

Compilation flags.

Source file. With a header-only approach only the main file is needed.

Flags to indicate cuda support.

Location of the "include" folder for your local cuda installation.

Location of the "lib64" folder for your local cuda installation.

Flag to suppress error messages raised from deprecated GPU architectures.

Compilation output file.

Dispatching a CUDA Job

```
// CUDA context

__global__ void bar(int arg0){
    // Every thread run this code block
}

// c++ context
void foo(){
    dim3 threads_per_block(32, 1, 1);
    dim3 blocks_per_grid(8, 1, 1);

    // Allocates 8 blocks where each block has 32 threads,
    // giving 8 * 32= 256 total threads.
    bar<<<blocks_per_grid, threads_per_block>>>(0);
}
```

CUDA Utilities

- `cudaMalloc()`
 - Allocates memory on the GPU
 - Must be invoked to create shared memory for a CUDA program
- `cudaMemcpy()`
 - Copies chunks of data on the GPU
 - Also used to transfer data between the GPU and CPU

Indexing as Control Flow

- Each CUDA thread receives the same copy of code instructions when a job is dispatched.
- To determine which thread does what, you must use the “index” of the thread.
- There are two index types available for us:
 - `threadIdx`
 - The index of the thread within a block. For a 32 thread block, the index values will be between 0 and 31 for all blocks.
 - Has 3 dimensions: {x, y, z}
 - `blockIdx`
 - The index of the block which contains the current thread.
 - Has 3 dimensions: {x, y, z}
 - x dimension index is of type long, while y and z indices are ints. This gives a maximum values of { $(2^{31}) - 1$, 65,535, 65,535 } for each one respectively.
- In practice it is often easier to just use the x dimensions alone. There is no performance benefit for using the other dimensions.

Synchronization

- Synchronization on the GPU is supported by limited
 - Synchronization between threads
 - `__syncthreads()` function allows for synchronization between threads within the same block. When invoked, all threads must reach the function before any may continue.
 - Synchronization between blocks
 - It is not possible to synchronize activity between blocks without using atomics.
 - Atomic synchronization
 - CUDA provides “atomic” functions that prevent race conditions when two blocks are interacting with the same memory location.
 - Can be used to synchronize activity between blocks.
 - Very slow, and should **never** be used under any circumstances.
- CPU Sync
 - The preferred way to synchronize between blocks is to return control back to the CPU.
 - `cudaDeviceSynchronize()` function forces CPU to wait until the CUDA program has finished.
 - The CPU can then call another CUDA program to interact with the synchronized data.

OOP and CUDA



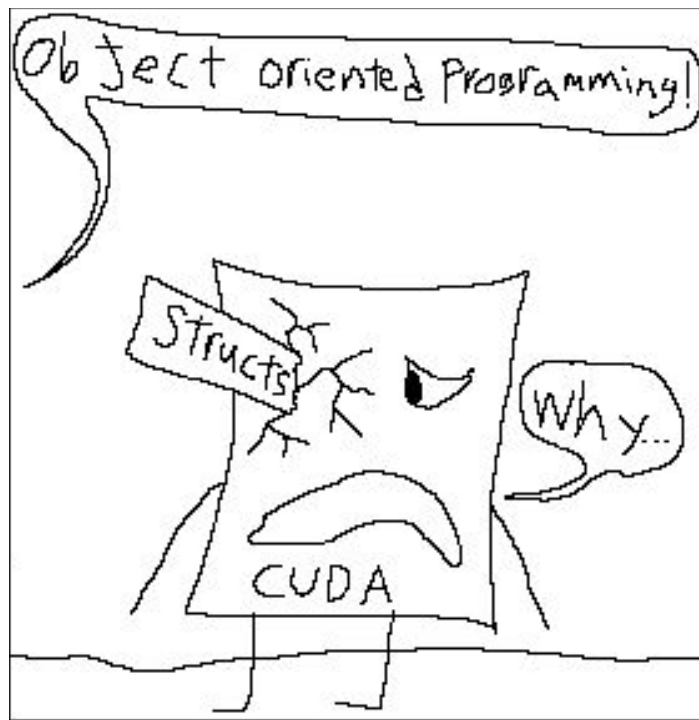
OOP and CUDA



OOP and CUDA



OOP and CUDA



OOP and CUDA



OOP and CUDA

- Classes are not allowed in CUDA, but they are allowed in C++.
 - This creates tension when a C++ application built on OOP design philosophies wants to use CUDA.
- Structs are allowed in CUDA
 - These structs can contain variables and functions just like a class.
 - Structs provide convenient data packages to pass between CUDA functions and serve as a very similar stand-in for classes.
 - No polymorphism
 - Constructor must be manually invoked.
- When compiling on Windows, the `__syncthreads()` function cannot be invoked directly inside a struct
 - Can get around this by passing the function into the struct at runtime using a function pointer.

Special Topics

- Efficient Atomics
- Cloud and server CUDA
- Allocation memory inside CUDA
- Popular libraries
 - thrust
 - cublas
 - cudnn
- NPU and TPU
- CUDA Manual

BeBar CUDA (b-cuda)

- Open source CUDA templates I created and have published under Apache 2.0 license.
- Header only
 - Simply drag and drop files directly into a project hierarchy.
- Includes the following:
 - **Indexer:** Useful functions for indexing multidimensional arrays in CUDA
 - **Math:** Math constants and utilities
 - **Transform:** Classes and functions for performing Vector math in CUDA.
 - **Quaternion:** Functions for operating on Vector4 quaternions and Vector3 euler angles
 - **Rigidbody:** Base class for a rigidbody in a physics simulation

<https://github.com/hatfield-c/b-cuda>

Case Study: Sum Array

- Sum all the elements of a given array
- Requires synchronization between threads and blocks to prevent race conditions.
- Solution:
 - Use a CUDA program that directs each thread to sum n number of elements in the array and store the results in the array position corresponding to thread 0 for each block.
 - Return to the CPU.
 - Dispatch a new CUDA call to sum the resulting elements using the same thread approach but with a stride.
 - Repeat until array is size 1.

Case Study: Search Array

- Search the elements of a given array for a particular value
- Requires synchronization between threads and blocks to prevent race conditions.
- Must keep tracks of the index locations in the array of any found instances.
- Solution:
 - Use a CUDA program that directs each thread to sum n number of elements in the array and store the results in a buffer.
 - Return to the CPU.
 - Dispatch a new CUDA call to sum the elements of the buffer using the same thread approach.
 - Repeat until array is size 1.

Case Study: Rigidbody Simulation

- Simulate a large number of spherical particles flying through space under the effects of gravity and with different starting conditions.
- Demonstrates how OOP can be used to simplify program architecture.